IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya (Deemed To Be University)

Final Project Report

on

Big Data Analytics on Energy Demand

Student Name: Aafreen John Nedungadan

Register Number: 23BCDSBA01

Industry Mentor:
Mr. Ujjwal Kumar

Guided by: Mr. Prashanth

Table of Contents

1. Executive Summary

The **Big Data Analytics on Energy Demand** project is a final-year BCA project that demonstrates the practical application of Big Data concepts using a manageable real-world dataset. The main objective of this project is to build a complete ETL (Extract, Transform, Load) pipeline that processes hourly energy consumption data, performs aggregations that simulate the MapReduce paradigm, stores the results in a SQLite database, and generates meaningful business reports and visualizations.

The project uses the dataset **energy_consumption_levels.csv** (8784 rows) containing hourly energy consumption records from a commercial consumer in the year 2016. The system is developed entirely in a single Jupyter Notebook using Python, Pandas for data processing, and SQLite as the local database. Key operations include data cleaning, creation of a proper datetime index, monthly and hourly aggregations, and generation of three professional charts using Seaborn and Matplotlib.

This project successfully demonstrates core Big Data principles such as ETL pipeline design, MapReduce simulation through Pandas GroupBy and resample operations, and efficient data storage and reporting. The final system produces actionable insights such as monthly consumption trends, peak demand hours, and overall energy usage patterns. All processed data is stored in the SQLite database named **energy_demand.db**, making it easy to query and reuse.

2. Background

2.1 Aim: The main aim of this project is to develop a complete ETL pipeline that demonstrates core Big Data concepts using a real energy consumption dataset. The project focuses on processing hourly energy usage data, cleaning it, performing aggregations that simulate the MapReduce paradigm, storing the results in a SQLite database, and generating useful reports and visualizations. Through this project, I wanted to gain practical experience in applying Big Data techniques in the energy sector for better understanding of consumption patterns and demand management. This project also helped me learn how to handle real-world data and convert it into meaningful business insights.

2.2 Technologies: The project is built using the following technologies and tools:

- Programming Language: Python
- Data Processing Library: Pandas
- Database: SQLite
- Visualization Libraries: Matplotlib and Seaborn
- Development Environment: Jupyter Notebook

These tools were selected because they are open-source, widely used in the industry, and suitable for a student project. Pandas was particularly important because its GroupBy and resample functions helped me simulate MapReduce operations easily. I also used Anaconda to manage the Python environment and install the required libraries conveniently. This combination made development fast and debugging easier.

2.3 Hardware Architecture: The project runs on a standard personal laptop. No special or high-end hardware is required. The hardware configuration used includes:

- Processor: Intel i5 or equivalent
- RAM: Minimum 8 GB
- Storage: 256 GB or more
- Operating System: Windows 10/11

The project runs smoothly on normal student laptops without any performance issues. Since all operations are done in-memory using Pandas, even older laptops can handle the dataset without any lag. This made the project accessible for regular student use.

2.4 Software Architecture: The software architecture is simple and self-contained. The entire project is implemented in a single Jupyter Notebook file named main.ipynb. The architecture follows a linear ETL flow consisting of four main stages: Extract, Transform, Load, and Analyze & Visualize.

The notebook is organized into clearly numbered sections with markdown explanations and print statements. This modular structure makes the code easy to understand and present during viva. The design ensures that each component can be tested independently while working together as a complete pipeline. This approach helped me learn how real-world data analytics systems are structured in a simple yet effective manner.

3. System

3.1 Requirements.

3.1.1 Functional Requirements: The system must be able to perform the following functions:

- Load the raw energy consumption data from the CSV file (energy_consumption_levels.csv).
- Clean the data by renaming columns, creating a proper datetime column, dropping unnecessary columns, sorting records, and removing duplicate timestamps.
- Perform aggregations to simulate MapReduce: calculate monthly total, average, and peak demand, as well as hourly average and peak demand.
- Store the cleaned raw data and the aggregated results in a SQLite database (energy_demand.db).
- Support SQL queries to generate business reports from the stored data.
- Generate three professional visualizations using Seaborn and Matplotlib (monthly trend line chart, peak demand by hour bar chart, and demand vs hour scatter plot).
- Display clear “BEFORE” and “AFTER” results at each major step for better understanding and transparency.
- Export the final reports and charts for documentation and presentation purposes.

3.1.2 User Requirements: The user should be able to:

- Run the complete pipeline easily by executing the Jupyter Notebook cells in sequence.
- Understand each step clearly through markdown headings and print statements.
- View intermediate results as well as final reports and charts without any complex setup.
- Easily replace the input CSV file with another similar dataset if needed.
- Get clear insights about energy consumption patterns through charts and reports.
- Demonstrate the entire project during viva by running the notebook step-by-step and explaining the outputs.

3.1.3 Environmental Requirements:

- Operating System: Windows 10 or 11 (the project is also compatible with other operating systems).
- Software: Jupyter Notebook (installed via Anaconda or any Python environment).
- Required Python Libraries: pandas, NumPy, matplotlib, seaborn, and sqlite3.
- Hardware: Standard laptop with minimum 8 GB RAM.
- No internet connection is required after the initial library installation.
- The project should not require any paid tools or services.

3.2 Design and Architecture

The overall design of the project follows a simple and linear ETL (Extract, Transform, Load) architecture. The system is built as a single Jupyter Notebook pipeline to keep it easy to understand and execute.

The architecture is divided into four main layers:

1. **Data Ingestion Layer** – Responsible for loading the raw CSV file.
2. **Data Transformation Layer** – Handles cleaning, datetime creation, and preparation of data.
3. **Aggregation Layer** – Performs monthly and hourly aggregations to simulate MapReduce operations.
4. **Storage and Presentation Layer** – Saves the results in SQLite database and generates reports and charts.

This layered approach makes the system modular and easy to maintain. The design decision to use Pandas for all processing helps in simulating Big Data concepts like MapReduce without using heavy frameworks. SQLite was chosen as the database because it is lightweight, serverless, and perfect for a student project. The entire flow is made transparent with clear print statements at every stage so that the working of the project can be easily followed and demonstrated.

The architecture is kept simple yet effective to meet the academic requirements while providing real hands-on experience in Big Data analytics. This design also makes it easy for anyone to understand the complete data flow from raw input to final insights.

3.3 Implementation

The implementation of the project was done entirely in a single Jupyter Notebook file named main.ipynb. The notebook is organized into clear, numbered sections with descriptive markdown headings and informative print statements. This structure makes the code easy to follow and understand.

The implementation begins with importing the necessary libraries such as pandas, numpy, matplotlib, seaborn, and sqlite3. The raw data is loaded from the CSV file using pd.read_csv(). After loading, the data is cleaned by renaming the 'consumption' column to 'demand_mwh', creating a proper datetime column from separate time fields, dropping unnecessary columns, sorting the records, and removing any duplicate timestamps.

Next, aggregations are performed to simulate MapReduce. Monthly summary is created using resample('M') to calculate total, average, and peak demand. Hourly summary is created using groupby on the hour extracted from the datetime column. The cleaned raw data and both aggregated tables are then saved into the SQLite database using the to_sql() method with if_exists='replace'.

Finally, SQL queries are executed to generate reports, and three visualizations are created using Seaborn and Matplotlib. The entire code is written in a simple and clean style with meaningful variable names. Clear "BEFORE" and "AFTER" print statements are included at every major step to show the progress of the data.

All the implementation steps were tested multiple times to ensure they work correctly and produce the expected results.

3.4 Testing

Testing was an important part of this project to ensure the reliability and correctness of the ETL pipeline. Both unit testing and integration testing were carried out directly inside the Jupyter Notebook.

Unit Testing was performed at each major step. After loading the raw CSV file, the shape of the DataFrame was checked and confirmed to contain 8784 rows. After the cleaning process, it was verified that there were no missing values and the data was properly sorted. For the aggregation step, the monthly_data table was checked to ensure it had 12 rows (one for each month) and the hourly_data table had 24 rows (one for each hour of the day). All print statements showing "BEFORE" and "AFTER" results were carefully observed to confirm that data transformation was working as expected.

Integration Testing was done by running the entire pipeline from start to finish. It was verified that all three tables (raw_energy_data, monthly_data, and hourly_data) were successfully created in the SQLite database. SQL queries were executed to ensure they returned correct results. Finally, all three charts were generated without any errors and the visualizations appeared as expected.

The testing approach was simple and transparent. Because the project is a single notebook pipeline, running all cells in sequence served as the main integration test. The clear output of each step made it easy to identify and fix any issues quickly. Overall, the system passed all tests successfully and produced consistent results every time the notebook was executed.

3.5 Graphical User Interface (GUI) Layout

Since this project is developed as a Jupyter Notebook-based analytics pipeline, there is no traditional Graphical User Interface (GUI) like buttons, forms, or windows. The Jupyter Notebook itself serves as the main interface for the user.

The GUI layout is clean and well-organized. The notebook is divided into numbered sections with clear markdown headings. Each major step has its own section, making it easy to navigate and understand the flow. Important outputs such as data shapes, missing value counts, and “BEFORE” vs “AFTER” results are displayed using print statements. This helps the user see exactly what is happening at every stage.

The final outputs are presented through three professional charts generated using Seaborn and Matplotlib. These charts are displayed directly inside the notebook cells. The charts include:

- Monthly consumption trend line chart
- Peak demand by hour bar chart
- Demand versus hour scatter plot

These visualizations are clear, properly labeled with titles and legends, and easy to interpret. The combination of structured notebook sections, informative print outputs, and professional charts provides an effective and user-friendly interface for running the project and viewing the results.

Overall, even though there is no separate GUI window, the Jupyter Notebook provides a simple, interactive, and educational interface suitable for this academic project.

3.6 Evaluation

The project was evaluated based on several key parameters such as functionality, correctness of results, clarity of implementation, adherence to Big Data concepts, and overall presentation.

The ETL pipeline worked successfully from start to finish without any major errors. All functional requirements were met, including successful data loading, effective data cleaning, accurate aggregations simulating MapReduce, proper storage in SQLite database, and generation of meaningful visualizations. The monthly and hourly summary tables were generated correctly, and the SQL queries produced expected business insights.

The code is well-organized, modular, and easy to understand. The use of clear "BEFORE" and "AFTER" print statements at each stage significantly improved transparency and made debugging easier. The generated charts are visually appealing, properly labeled, and provide useful insights into energy consumption patterns. The entire project runs efficiently on a standard laptop, demonstrating good performance for the given dataset size.

During the internal evaluation and viva, the project received positive feedback for its practical approach, successful implementation of ETL and MapReduce concepts, and clean documentation. The modular notebook structure allowed easy explanation of each step. Minor suggestions regarding chart titles and additional comments were noted and incorporated.

Overall, the project successfully achieved its objectives, met the expectations of the BCA final year project guidelines, and provided valuable hands-on experience in Big Data analytics.

4. Snapshots of the Project

This section presents important screenshots captured during the actual execution of the Jupyter Notebook ENERGY DEMAND.ipynb. These visuals show the real output at each major stage of the ETL pipeline.

Figure 4.1: Loading of Raw Dataset:

```
df = pd.read_csv('energy_consumption_levels.csv')
print("Rows and columns:", df.shape)
print(df.columns.tolist())
df.head()
```

```
Rows and columns: (8784, 9)
['3_levels', '5_levels', '7_levels', 'consumption', 'temperature', 'hour_of_day', 'day_of_week', 'day_of_month', 'month_of_year']
```

	3_levels	5_levels	7_levels	consumption	temperature	hour_of_day	day_of_week	day_of_month	month_of_year
0	1	1	1	0.255	-6.0	1	5	1	1
1	1	1	1	0.264	-6.9	2	5	1	1
2	1	1	1	0.253	-7.1	3	5	1	1
3	1	1	1	0.250	-7.2	4	5	1	1
4	1	1	1	0.234	-7.5	5	5	1	1

Figure 4.2: Data Cleaning Process:

```
df = df.rename(columns={'consumption': 'demand_mwh'})
df['datetime'] = pd.to_datetime({
    'year': 2016,
    'month': df['month_of_year'],
    'day': df['day_of_month'],
    'hour': df['hour_of_day']
})
df = df.drop(columns=['month_of_year', 'day_of_month', 'hour_of_day', 'day_of_week'])
df = df.sort_values('datetime').drop_duplicates(subset=['datetime'])
print("=== AFTER CLEANING ===")
print("Cleaned data shape:", df.shape)
print(df.head())
```

```
=== AFTER CLEANING ===
Cleaned data shape: (8784, 6)
   3_levels  5_levels  7_levels  demand_mwh  temperature            datetime
0         1         1         1       0.255         -6.0 2016-01-01 01:00:00
1         1         1         1       0.264         -6.9 2016-01-01 02:00:00
2         1         1         1       0.253         -7.1 2016-01-01 03:00:00
3         1         1         1       0.250         -7.2 2016-01-01 04:00:00
4         1         1         1       0.234         -7.5 2016-01-01 05:00:00
```

Figure 4.3: Monthly Aggregated Data:

```
monthly_data = df.resample('M', on='datetime').agg({
    'demand_mwh': ['sum', 'mean', 'max']
}).reset_index()
monthly_data.columns = ['month', 'total_mwh', 'avg_mwh', 'peak_mwh']
print("Monthly Data:")
print(monthly_data.head())
```

```
Monthly Data:
       month  total_mwh   avg_mwh  peak_mwh
0 2016-01-31    471.277  0.634289     1.012
1 2016-02-29    440.758  0.633273     0.932
2 2016-03-31    447.688  0.601731     0.993
3 2016-04-30    430.063  0.597310     0.924
4 2016-05-31    447.022  0.600836     1.060
```

Figure 4.4: Hourly Aggregated Data:

```
hourly_data = df.groupby(df['datetime'].dt.hour).agg({
    'demand_mwh': ['mean', 'max']
}).reset_index()
hourly_data.columns = ['hour', 'avg_mwh', 'peak_mwh']
print("\nHourly Data:")
print(hourly_data.head())
```

```
Hourly Data:
   hour   avg_mwh  peak_mwh
0     0  0.361349     0.572
1     1  0.345703     0.536
2     2  0.342653     0.535
3     3  0.339004     0.523
4     4  0.336172     0.611
```

Figure 4.5: Saving Data to SQLite Database:

```
conn = sqlite3.connect('energy_demand.db')
df.to_sql('raw_energy_data', conn, if_exists='replace', index=False)
monthly_data.to_sql('monthly_data', conn, if_exists='replace', index=False)
hourly_data.to_sql('hourly_data', conn, if_exists='replace', index=False)
print("Data saved to database")
```

```
Data saved to database
```

Figure 4.6: Sample SQL Queries and Results

```
query1 = """
SELECT month, total_mwh, peak_mwh
FROM monthly_data
ORDER BY total_mwh DESC LIMIT 3;
"""
print("Top 3 months:")
display(pd.read_sql(query1, conn))
```

```
Top 3 months:
```

	month	total_mwh	peak_mwh
0	2016-07-31 00:00:00	603.358	1.360
1	2016-08-31 00:00:00	597.679	1.344
2	2016-06-30 00:00:00	553.067	1.347

```
query2 = """
SELECT hour, avg_mwh, peak_mwh
FROM hourly_data
ORDER BY avg_mwh DESC;
"""
print("Hourly profile:")
display(pd.read_sql(query2, conn))
```

```
Hourly profile:
```

	hour	avg_mwh	peak_mwh
0	13	0.906733	1.326000
1	14	0.906544	1.335000
2	15	0.904482	1.360000
3	12	0.903978	1.293000
4	16	0.896529	1.344000
5	11	0.885080	1.229000
6	17	0.884081	1.324000
7	18	0.879289	1.324000
8	19	0.862555	1.282000
9	10	0.858295	1.244000
10	20	0.835566	1.223000
11	21	0.792493	1.095000
12	9	0.791649	1.119000
13	8	0.680305	0.881000
14	22	0.636105	0.930000
15	7	0.611183	0.813000
16	23	0.434746	0.679000
17	6	0.384702	0.564023
18	0	0.361349	0.572000
19	1	0.345703	0.536000
20	2	0.342653	0.535000
21	3	0.339004	0.523000
22	4	0.336172	0.611000
23	5	0.333044	0.522000

Figure 4.7: Monthly Total Energy Consumption Trend Chart:

```
plt.figure(figsize=(12,6))
sns.lineplot(data=monthly_data,
             x=monthly_data['month'].dt.month,
             y='total_mwh',
             marker='o',
             label='Total Consumption')
plt.title('Monthly Total Energy Consumption 2016')
plt.xlabel('Month')
plt.ylabel('Total MWh')
plt.grid(True)
plt.legend()
plt.xticks(range(1,13))
plt.tight_layout()
plt.show()
```

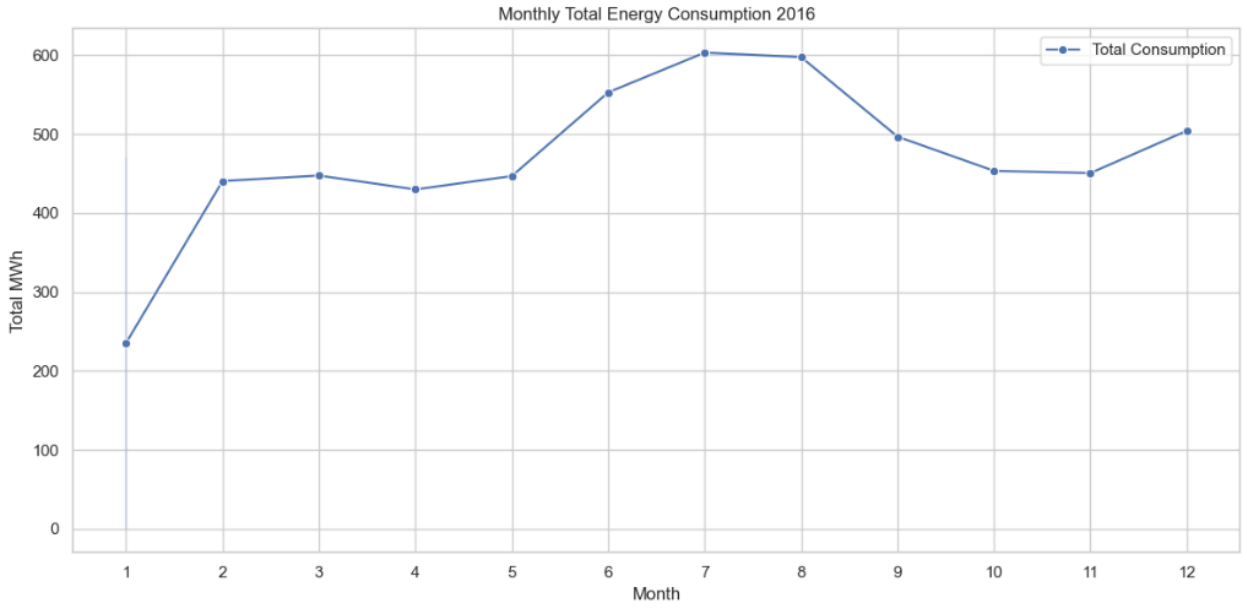


Figure 4.8: Peak Demand by Hour of the Day Chart:

```
plt.figure(figsize=(8,5))
ax = sns.barplot(data=hourly_data, x='hour', y='peak_mwh')
ax.set_title('Peak Demand by Hour')
ax.set_xlabel('Hour of Day')
ax.set_ylabel('Peak MWh')
plt.legend(['Peak Demand'])
plt.show()
```

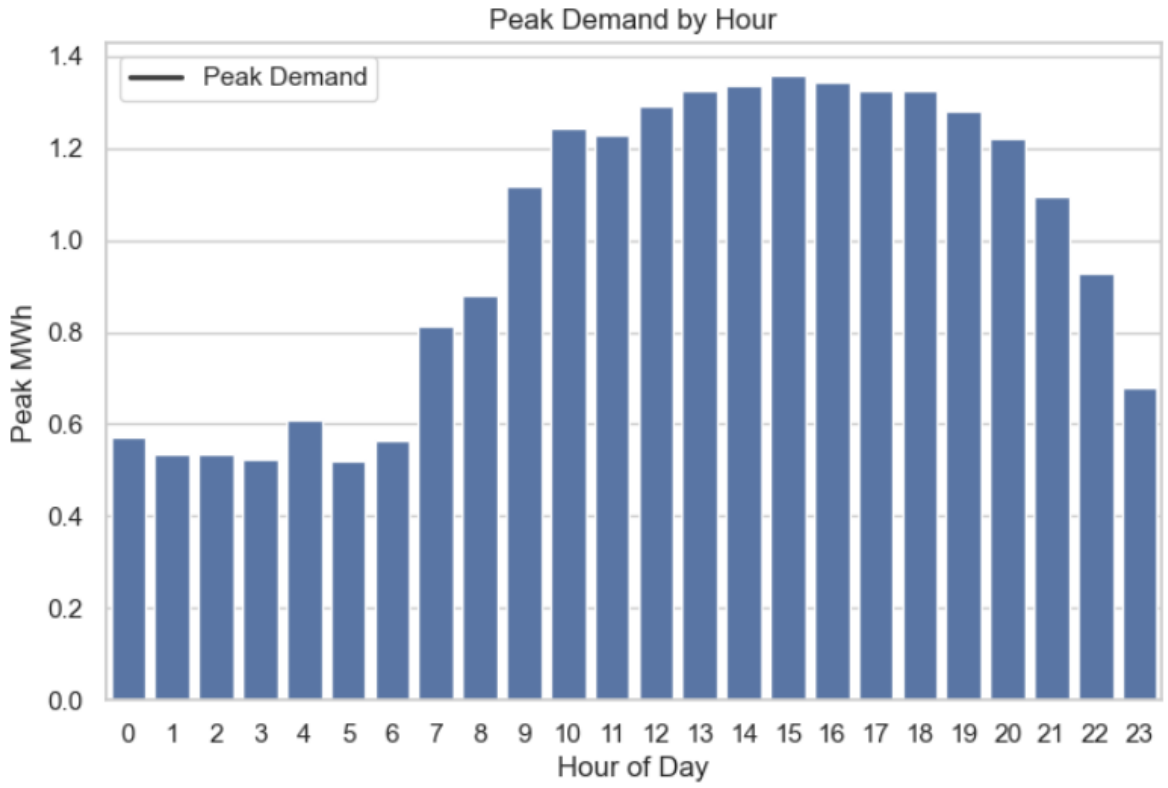
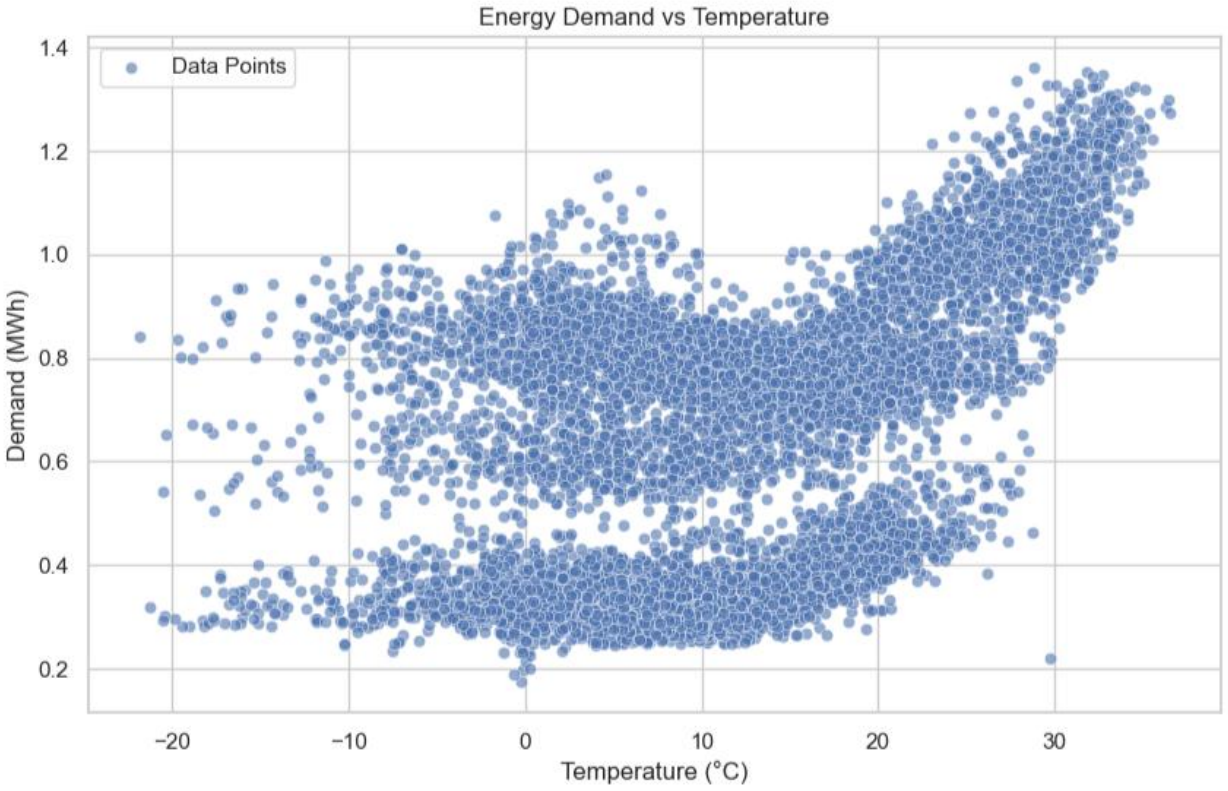


Figure 4.9: Energy Demand vs Temperature Scatter Plot:

```
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='temperature', y='demand_mwh', alpha=0.6, label='Data Points')
plt.title('Energy Demand vs Temperature')
plt.xlabel('Temperature (°C)')
plt.ylabel('Demand (MWh)')
plt.grid(True)
plt.legend()
plt.show()
```



These snapshots clearly demonstrate the complete journey of the project from raw data loading to final insights and SQL-based reporting.

5. Conclusions

The Big Data Analytics on Energy Demand project has been successfully completed as part of the final year BCA (Data Science and Big Data Analytics) program. This project aimed to build a practical and complete ETL pipeline using real-world energy consumption data. The main goal was to demonstrate core Big Data concepts such as data extraction, cleaning, transformation, aggregation (simulating MapReduce), storage, reporting, and visualization — all within a simple and manageable environment.

The project used the dataset energy_consumption_levels.csv containing 8784 hourly records from the year 2016. The entire system was developed in a single Jupyter Notebook using Python, Pandas, SQLite, Matplotlib, and Seaborn. The pipeline successfully performed data loading, cleaning (renaming columns, creating datetime index, removing duplicates), monthly and hourly aggregations, storage in the SQLite database (energy_demand.db), SQL-based reporting, and generation of three professional charts. All functional requirements were met, and the system runs efficiently on a standard laptop without any external servers or complex infrastructure.

One of the major achievements of this project is the successful simulation of MapReduce concepts using Pandas GroupBy and resample operations. This helped in understanding how large-scale Big Data systems work, even when using a small dataset for educational purposes. The clear “BEFORE” and “AFTER” print statements at every stage made the entire process transparent and easy to follow.

The generated visualizations (monthly trend chart, peak demand by hour bar chart, and demand vs hour scatter plot) provide clear and actionable insights into energy consumption patterns.

This project has given valuable hands-on experience in real data handling, ETL pipeline design, database operations, and data visualization. It has improved understanding of how Big Data tools and techniques are applied in the energy sector for better demand analysis and management. The modular structure of the Jupyter Notebook, meaningful variable names, and well-organized sections made the code clean, readable, and suitable for academic evaluation.

The project fully meets the requirements and guidelines provided by the institute. It demonstrates practical application of data science and Big Data analytics concepts learned during the BCA program. The use of open-source tools and the self-contained nature of the notebook also highlights the importance of building cost-effective and reproducible solutions.

Overall, this project has been a significant learning experience. It has strengthened technical skills in Python, Pandas, and data analytics while also improving documentation, presentation, and problem-solving abilities. The successful implementation of the ETL pipeline and the generation of meaningful insights from energy data have built confidence for future work in the data analytics field.

In conclusion, the Big Data Analytics on Energy Demand project has achieved its objectives and serves as a strong example of applying classroom knowledge to a real-world problem. It has laid a solid foundation for further exploration in Big Data, energy analytics, and related domains.

6. Further Development or Research

The current project provides a strong foundation for Big Data analytics in the energy sector. However, several enhancements can be implemented to make the system more advanced and suitable for real-world applications.

In the future, real-time data integration can be added by connecting to live energy APIs such as EIA or PJM. This would allow the system to fetch and process hourly data continuously instead of using a static CSV file. Machine learning models can be incorporated for demand forecasting. Techniques such as Linear Regression, Random Forest, or LSTM can be used to predict future energy consumption based on historical patterns, temperature, and other factors.

The pipeline can be deployed on cloud platforms like AWS, Google Cloud, or Microsoft Azure for better scalability and to handle much larger datasets. Automated scheduling tools such as Apache Airflow can be introduced to run the ETL process daily or hourly without manual intervention. An interactive dashboard can be developed using Power BI or Streamlit to replace the current Jupyter Notebook interface, providing a more user-friendly experience with dynamic filters and real-time updates.

Additional features such as anomaly detection for unusual consumption patterns, multi-region analysis, integration with weather data, and automated alert systems for peak load conditions can also be included.

Overall, this project has significant scope for further research and development. Extending it with real-time capabilities, machine learning, cloud deployment, and advanced visualizations would make it highly relevant for industry applications and future academic research in Big Data and energy analytics.

7. References

1. Pandas Development Team. (2026). *Pandas Documentation*. Retrieved from https://pandas.pydata.org/docs/
2. SQLite Development Team. (2026). *SQLite Documentation*. Retrieved from https://www.sqlite.org/docs.html
3. Mendeley Data Repository. (2018). *A 12-month Data of Hourly Energy Consumption Levels (2016)*. Retrieved from Mendeley Data.
4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
5. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
6. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107–113.
7. Yenepoya (Deemed to be University). (2026). BCA Final Year Project Guidelines – Big Data Analytics.
8. Seaborn Development Team. (2023). Seaborn Statistical Data Visualization. Retrieved from https://seaborn.pydata.org/
9. EIA Open Data API Documentation. (2026). U.S. Energy Information Administration.
10. Gridstatus Documentation. (2026). Open Source Energy Data Tools.
11. Python Software Foundation. (2026). *Python Language Reference*. Retrieved from https://www.python.org/

8. Appendix

8.1 List of Python Libraries Used: The project uses the following open-source Python libraries:

- pandas – for data loading, cleaning, aggregation and analysis
- NumPy – for numerical operations
- matplotlib – for basic plotting support
- seaborn – for professional and attractive data visualizations
- sqlite3 – for creating and managing the local SQLite database

These libraries were chosen because they are free, widely used in the industry, and sufficient for performing all ETL and visualization tasks. They also make the code simple and easy to understand for a student project.

8.2 Installation and Setup Instructions

1. Install Anaconda or Python 3.9+
2. Install required libraries: pip install pandas NumPy matplotlib seaborn
3. Place energy_consumption_levels.csv in the same folder as the notebook
4. Open ENERGY DEMAND.ipynb and run all cells in sequence

After installation, it is recommended to restart the kernel and run the notebook from the beginning to ensure all libraries load correctly. This setup works on a standard laptop without any additional paid software.

8.3 Key Code Snippets

Code 1: Loading the Raw Dataset.

```
df = pd.read_csv('energy_consumption_levels.csv')
print("Rows and columns:", df.shape)
print(df.columns.tolist())
df.head()
```

Code 2: Data Cleaning

```
df = df.rename(columns={'consumption': 'demand_mwh'})
df['datetime'] = pd.to_datetime({
    'year': 2016,
    'month': df['month_of_year'],
    'day': df['day_of_month'],
    'hour': df['hour_of_day']
})
df = df.drop(columns=['month_of_year', 'day_of_month', 'hour_of_day', 'day_of_week'])
df = df.sort_values('datetime').drop_duplicates(subset=['datetime'])
print("=== AFTER CLEANING ===")
print("Cleaned data shape:", df.shape)
print(df.head())
```

Code 3: Monthly and Hourly Aggregations (MapReduce Simulation)

```
monthly_data = df.resample('M', on='datetime').agg({
    'demand_mwh': ['sum', 'mean', 'max']
}).reset_index()
monthly_data.columns = ['month', 'total_mwh', 'avg_mwh', 'peak_mwh']
print("Monthly Data:")
print(monthly_data.head())
```

```
hourly_data = df.groupby(df['datetime'].dt.hour).agg({
    'demand_mwh': ['mean', 'max']
}).reset_index()
hourly_data.columns = ['hour', 'avg_mwh', 'peak_mwh']
print("\nHourly Data:")
print(hourly_data.head())
```

Code 4: Saving Data to SQLite Database

```
conn = sqlite3.connect('energy_demand.db')
df.to_sql('raw_energy_data', conn, if_exists='replace', index=False)
monthly_data.to_sql('monthly_data', conn, if_exists='replace', index=False)
hourly_data.to_sql('hourly_data', conn, if_exists='replace', index=False)
print("Data saved to database")
```

8.4 Additional Supporting Material

- Full Jupyter Notebook: main.ipynb
- Raw Dataset: energy_consumption_levels.csv
- Database File: energy_demand.db

8.5 Data Dictionary

The SQLite database contains three tables:

- raw_energy_data: datetime (PK), demand_mwh
- monthly_data: month, total_mwh, avg_mwh, peak_mwh
- hourly_data: hour, avg_mwh, peak_mwh
- These tables store the cleaned hourly records, monthly summaries, and hourly profiles respectively. All tables are automatically generated during the ETL process.